CLA 4 – DETAILED OBSERVATIONS REPORT

ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle train.csv)

Model: Random Forest Classifier | Test Accuracy: 81.56% (0.8156)

1. Introduction

Machine learning model deployment involves making a trained model available for predictions on new data. After training, the model can be used in different inference modes depending on how records arrive and how quickly predictions are needed. This assignment demonstrates online inference, batch inference, streaming inference, and REST API-based model serving using FastAPI. The Titanic training dataset is used to build a classification pipeline that predicts whether a passenger survived.

2. Objective

The objective is to develop a machine learning classification model, evaluate its performance, save the complete preprocessing and prediction pipeline using Joblib, demonstrate three inference strategies using unseen passenger records, and expose the same saved model through a FastAPI application. The API is tested using Swagger UI and a Python client, including validation of an invalid request.

3. Dataset Description

The assignment specifies the Titanic – Machine Learning from Disaster dataset available through Kaggle and requires the labelled train.csv file. The target column is Survived, where 0 represents did not survive and 1 represents survived. The model uses the following passenger features: Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked.

Pclass represents passenger class; Sex represents the recorded passenger sex; Age is passenger age; SibSp and Parch represent family relationships aboard; Fare is the ticket fare; and Embarked indicates the port of embarkation. Only the specified dataset is used for this assignment.

4. Data Preprocessing and Model Training

The labelled data is split into training and testing subsets using an 80:20 split, with stratification to preserve the target-class proportions. Preprocessing is included in the same pipeline as the classifier so that the same transformations are applied consistently during every inference strategy.

For numerical features (Pclass, Age, SibSp, Parch, and Fare), missing values are filled using the median. A standard scaler is then applied. For categorical features (Sex and Embarked), missing values are filled using the most frequent category and One-Hot Encoding converts categories into model-readable numeric features. The encoder is configured to ignore previously unseen categories instead of failing.

A Random Forest Classifier with 200 estimators and a fixed random state is trained on the training subset. The pipeline is evaluated on the held-out test subset using accuracy. The obtained test accuracy is 0.8156, or 81.56%. The complete fitted pipeline, including preprocessing and the classifier, is saved to model.pkl using Joblib.

5. Online Inference

Online inference produces a prediction for one passenger record at a time. The online_inference.py program loads model.pkl, selects one record from unseen_records.csv, passes the passenger features through the saved pipeline, and prints the predicted class. A prediction of 1 is displayed as “Survived”, while a prediction of 0 is displayed as “Did not survive”. This demonstrates an immediate, single-record prediction without retraining the model.

6. Batch Inference

Batch inference is used when multiple records can be processed together. The batch_inference.py program selects 20 held-out passenger records and sends their feature values to the same saved pipeline in one prediction operation. The predicted class and a readable label are added to the results. The output is saved in batch_predictions.csv, which can be opened in spreadsheet software for inspection.

7. Streaming Inference

Streaming inference simulates records arriving over time. The streaming_inference.py program selects 10 held-out passenger records and processes them one by one. A one-second delay is inserted between records to simulate the arrival of incoming data. The prediction for each record is printed as it is processed. This differs from batch inference because the records are handled sequentially rather than submitted together as one batch.

8. Model Serving using FastAPI

FastAPI exposes the saved model through HTTP endpoints so another application can request predictions. The app.py application loads model.pkl and defines two endpoints. GET /health returns the API health status and indicates that the model is loaded. POST /predict accepts passenger details, converts the request into a data frame with the expected feature columns, and returns both the numeric prediction and a human-readable label.

Pydantic validates incoming request fields before prediction. For example, Pclass must be between 1 and 3, Sex must be male or female, and numerical fields are checked against the defined constraints. Swagger UI at /docs provides an interactive interface for sending requests and viewing responses. The API was started using Uvicorn.

9. API and Client Testing

A valid POST /predict request containing passenger details returned HTTP status code 200 and a prediction response. An invalid request with Pclass set to 5 returned HTTP status code 422 (Unprocessable Content), showing that the request validation rejects an out-of-range passenger class. The Python client in client.py also sent a request to the prediction endpoint and received status code 200 with a prediction and label.

10. Observations

10.1 Online Inference vs Batch Inference

Online inference predicts one record at a time, usually when a request arrives, and is useful when an immediate response is needed. Batch inference processes a collection of records together and is useful for bulk or scheduled prediction jobs. In this assignment, online inference uses one unseen passenger while batch inference processes 20 unseen passengers and saves their predictions to a CSV file.

10.2 Streaming Inference vs Batch Inference

Streaming inference processes records continuously as they arrive, one record at a time. Batch inference waits until a group of records is available and processes the group together. The streaming demonstration processes 10 records with a one-second delay, while the batch demonstration predicts 20 records in a single operation.

10.3 Why FastAPI is Used for Model Serving

FastAPI provides a straightforward way to expose machine learning predictions through REST API endpoints. It supports request validation with Pydantic, returns structured JSON responses, and provides interactive API documentation through Swagger UI. This allows the model to be tested or consumed by other programs without embedding the model code in each client.

10.4 Why the Same Trained Model Must Be Reused

Using the same saved pipeline for online, batch, streaming, and API predictions ensures that preprocessing and model behaviour remain consistent. It prevents differences caused by retraining separate models for each strategy and saves time and computational resources. The assignment therefore trains the model once and reloads model.pkl for all inference tasks.

11. Results Summary

Task	Demonstration	Output
Model training	80:20 train/test split; Random Forest pipeline	model.pkl; accuracy 81.56%
Online inference	One held-out passenger	Single prediction displayed
Batch inference	20 held-out passengers	batch_predictions.csv
Streaming inference	10 records, one-second delay	Predictions displayed sequentially
FastAPI health	GET /health	Health response
FastAPI prediction	POST /predict with valid input	HTTP 200 and prediction
Validation test	POST /predict with Pclass = 5	HTTP 422 validation error
Python client	client.py calls prediction endpoint	HTTP 200 and prediction response

12. Conclusion

The assignment demonstrates the end-to-end workflow of training, saving, reusing, and serving a machine learning classification pipeline. The Random Forest pipeline achieved a test accuracy of 81.56%. Online, batch, and streaming inference were completed using held-out passenger records. The saved model was exposed through FastAPI, a valid request returned a prediction, an invalid request was rejected by Pydantic validation, and the Python client received a successful response. These results demonstrate how one trained model can support multiple inference strategies and REST API serving.